

FedDQN: Federated Deep Q-Network for Rerouting in LEO Satellite Networks

Su Li, He Sun, Qing Chen, Xiaoxiang Wang*

Beijing University of Posts and Telecommunications, Beijing, 100876, China
samlee@bupt.edu.cn, 2023210376@bupt.edu.cn, QingChen@bupt.edu.cn, *cpwang@bupt.edu.cn

Abstract—In dynamic satellite networks, rapid topology changes and resource constraints pose significant challenges for efficient rerouting decisions. Traditional routing algorithms struggle to adapt to the high-dynamic nature of Low Earth Orbit (LEO) satellite constellations, while centralized deep reinforcement learning (DRL) approaches suffer from high communication overhead and privacy concerns. In this paper, we propose a Federated Deep Q-Network (FedDQN) algorithm that combines the decision-making capabilities of DRL with the distributed training paradigm of federated learning. We introduce three key contents: a semi-asynchronous federated aggregation strategy that reduces idle waiting time by selecting only a subset of active nodes per round; a Dueling Deep Q-Network (DQN) architecture that decomposes Q-values into state-value and advantage functions for more accurate policy evaluation; and a prioritized experience replay mechanism that enhances sampling efficiency for critical network events. Experimental results on a Walker Delta 288/12/1 constellation demonstrate that FedDQN reduces cumulative communication overhead by 70.8% compared to the standard DQN and by 61.0% compared to Actor-Critic (A2C), while maintaining robust convergence properties under various failure scenarios.

Keywords—low earth orbit, federated learning, satellite clustering, rerouting optimization

I. INTRODUCTION

Low Earth Orbit (LEO) satellite networks have emerged as a critical infrastructure for global seamless communication, offering advantages such as low transmission latency, wide-area coverage, and rapid deployment capabilities[1]. However, the inherent high-dynamic topology characteristics of LEO networks, combined with stringent on-board resource constraints, present formidable challenges for real-time and effective rerouting algorithms. Specifically, the periodic motion of satellite nodes leads to continuous network topology changes and frequent link state updates, while limited computational resources, storage capacity, and energy supply on satellites hinder the execution of complex algorithms in real-time[2].

Traditional routing protocols, such as OSPF and Dijkstra's shortest path algorithm, rely on static topology information and pre-computed routing tables, which are ill-suited for the dynamic nature of satellite networks. These approaches lack the adaptability to respond to rapid topology changes and cannot optimize for multiple objectives such as latency, reliability, and load balancing simultaneously[3].

Deep Reinforcement Learning (DRL) has shown remarkable success in sequential decision-making tasks, with Deep Q-Network(DQN) being one of the most representative algorithms[4]. However, applying standard DQN directly to LEO satellite network rerouting faces three fundamental limitations: uniform random sampling from the experience

replay buffer leads to inefficient utilization of high-value transition samples, slowing down policy convergence[5]; high correlation between adjacent experience samples identically distributed (IID) assumption typically desired for stable neural network training, leading to parameter update oscillations[6]; the max operator in Q-learning amplifies noise, leading to systematic positive bias in Q-value estimation and affecting policy selection accuracy[7].

Experience replay is a core component of the Deep Q-Network (DQN) algorithm. By storing experience tuples (s, a, r, s') generated from agent-environment interactions and employing random sampling for training, it effectively breaks temporal correlations among data points, thereby improving sample efficiency. However, conventional uniform sampling strategies assign equal probability to all experience samples, neglecting the variations in information gain that different samples contribute to network parameter updates[8].

In the context of satellite network routing, the network topology remains relatively stable; thus, the majority of experience samples correspond to routine routing decisions. These samples typically exhibit small Temporal Difference (TD) errors and contribute marginally to network parameter updates. Conversely, experience samples stemming from rare events—such as link failures and orbit handovers—are scarce yet contain critical state transition information, making them essential for enhancing policy robustness[9]. Consequently, the uniform sampling mechanism causes high-value samples to be overwhelmed by a preponderance of low-value samples, resulting in inefficient experience buffer utilization and sluggish convergence.

To address these challenges, we propose Federated Deep Q-Network (FedDQN), a novel algorithm that integrates federated learning with DQN optimization. The main works of this paper are as follows:

- We introduce a horizontal federated learning mechanism where only $p \times N$ nodes participate in each aggregation round, significantly reducing idle waiting time and communication overhead.
- We decompose the Q-value function into state-value $V(s)$ and advantage $A(s, a)$ functions, enabling decoupled and fine-grained value estimation that eliminates action selection bias.
- We employ a priority-based sampling mechanism where sample priority is calculated as $P_i = (|\delta_i| + \epsilon)^\alpha$, enhancing the sampling probability of high-value experiences.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. Network Topology Model

We model the LEO satellite network as a time-varying directed weighted graph $G(V, E, t)$, where E denotes the set of Inter-Satellite Links and $V = \{v_1, \dots, v_N\}$ represents the set of $N = 288$ satellite nodes. Fig. 1 illustrates the network follows a Walker Delta 288/12/1 constellation configuration with orbital altitude $h = 550$ km, inclination $i = 53^\circ$, 12 orbital planes, and 24 satellites per plane[10].

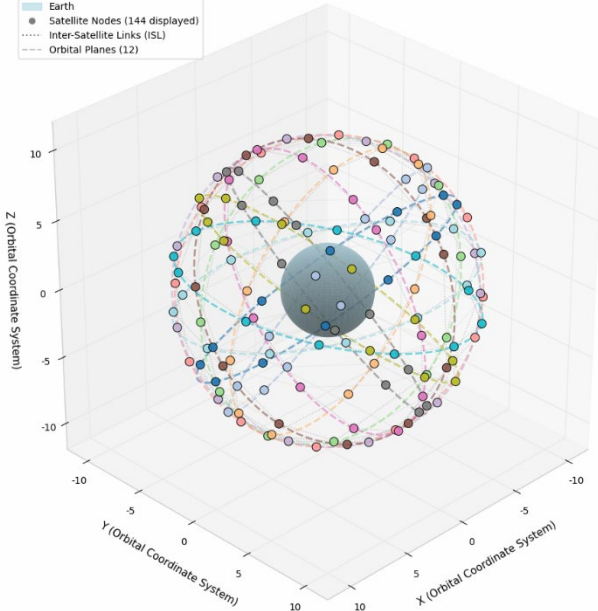


Fig. 1. Network Topology and Rerouting Scenario in LEO Constellation

B. Rerouting Optimization Objectives

The rerouting problem is formulated as a multi-objective optimization:

$$\min_{\pi} \mathcal{F}(\pi) = [\alpha_1 f_1(\pi), \alpha_2 f_2(\pi), \alpha_3 f_3(\pi)] \quad (1)$$

where $f_1(\pi)$ denotes end-to-end latency minimization, $f_2(\pi)$ denotes packet loss rate minimization, $f_3(\pi)$ denotes high-load node count minimization, $\alpha_1 = 0.4, \alpha_2 = 0.35, \alpha_3 = 0.25$ are weight coefficients.

C. Markov Decision Process Formulation

We formulate the rerouting problem as an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

For state space $\mathcal{S}$, $s_t = [\mathbf{T}_t^{\text{local}}, \mathbf{L}_t, \mathbf{Q}_t, \mathbf{N}_t, v_d, v_s]$. $\mathbf{T}_t^{\text{local}}$ is the local topology matrix, $\mathbf{L}$ is link quality vector, $\mathbf{Q}$ is queue length vector, and $\mathbf{N}$ is neighbor node status.

For action space $\mathcal{A}$, $\mathcal{A}(s_t) = \{a_j | v_j \in \mathcal{N}(i), c_{ij}(t) = 1\}$, representing the selection of next-hop forwarding nodes.

For reward $\mathcal{R}$, $\mathcal{R}(s_t, a_t) = r_{\text{delay}} + r_{\text{loss}} + r_{\text{handover}}$, each component represents rewards for successful transmission and penalties for delay, loss, and handover events respectively.

D. Detailed Description of Core Optimization Objectives

1) Minimization of Average End-to-End Delay

End-to-end delay is a fundamental metric for evaluating routing performance, directly affecting user experience and the quality of real-time services, the optimization objective formula is:

$$f_1(\pi) = \frac{1}{|F|} \sum_{f \in F} D(s_f, d_f, \pi) \quad (2)$$

where F is the set of all data flows in the network, $|F|$ is the number of data flows, and s_f and d_f are the source and destination nodes of flow f , respectively.

2) Minimization of Service Packet Loss Rate

Packet loss rate is a key metric for measuring network reliability, particularly crucial in scenarios involving satellite link failures or congestion. Packet loss primarily stems from two sources: link failure packet loss, occurring when data packets in transmission are lost due to ISL interruptions caused by satellite movement or failures; queue overflow packet loss, occurring when newly arrived data packets are discarded because the link queue length exceeds buffer capacity.

Define the loss of e_{ij} as $L_{ij}(t) = L_{ij}^{\text{fail}}(t) + L_{ij}^{\text{overflow}}(t)$. The optimization objective for packet loss rate is:

$$f_2(\pi) = \frac{\sum_{e_{ij} \in E} \int_0^T L_{ij}(t) \cdot \Lambda_{ij}(t) dt}{\sum_{e_{ij} \in E} \int_0^T \Lambda_{ij}(t) dt} \quad (3)$$

where $\Lambda_{ij}(t)$ is the data transmission rate.

3) Minimization of the Number of High-Load Nodes

Load balancing is a significant means to enhance network survivability. When some nodes are overloaded, it not only increases the risk of delay and packet loss but also makes these nodes vulnerable points of the network; once they fail, network connectivity will be severely affected. Define the load degree of node v_i as:

$$\rho_i(t) = \frac{\sum_{j \in \mathcal{N}(i)} \Lambda_{ij}(t)}{\sum_{j \in \mathcal{N}(i)} B_{ij}(t)} \quad (4)$$

where $\mathcal{N}(i)$ is the set of neighbor nodes of node v_i . When $\rho_i(t) > \rho_{\text{th}}$, node v_i is considered to be in a high-load state, where $\rho_{\text{th}} = 0.7$ is the load threshold. The optimization objective for the number of high-load nodes is:

$$f_3(\pi) = \frac{1}{T} \int_0^T \sum_{i=1}^N \mathbb{I}(\rho_i(t) > \rho_{\text{th}}) dt \quad (5)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

III. FEDDQN ALGORITHM DESIGN

FedDQN aims to achieve distributed, high-efficiency, and secure optimization of satellite network routing decisions through the deep integration of a federated learning architecture and an improved DRL model. As can be observed from Fig. 2, the core design of FedDQN lies in constructing a hierarchical collaborative training framework. While ensuring data localization at each satellite node, it achieves the collaborative evolution of the global policy through the aggregation and sharing of model parameters, and employs Dueling DQN and Prioritized Experience Replay to optimize the decision model of the standard DQN.

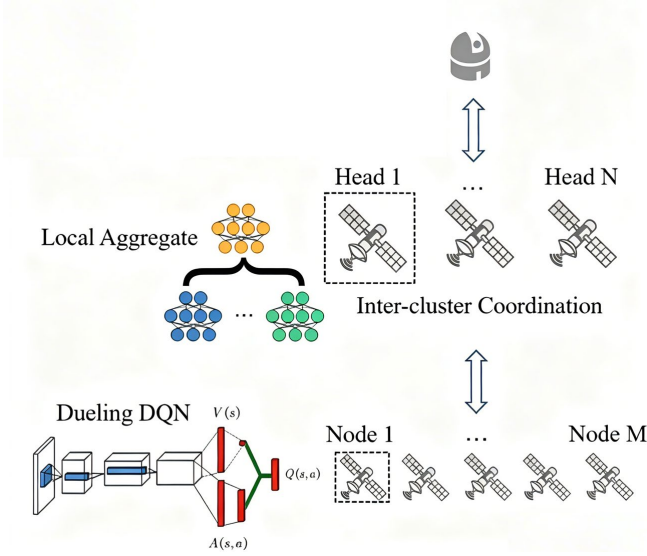


Fig. 2. FedDQN Algorithm Structure Diagram

A. Federated Learning Framework

The Federated Learning framework for satellite networks employs a "server-client" architecture, mapping the traditional centralized FL paradigm onto the physical topology of the satellite constellation. Within this architecture, cluster head satellites serve as aggregation servers, responsible for collecting, aggregating, and distributing global model parameters. Regular satellite nodes act as clients, performing model training using locally observed network state data and uploading the resulting model parameters to the server.

The FL framework comprises three logical layers: the Application Layer, the Coordination Layer, and the Execution Layer. The Application Layer handles task scheduling, global model initialization, and training result visualization. The Coordination Layer, deployed on cluster head nodes, manages local aggregation and inter-cluster communication coordination. The Execution Layer, deployed on all satellite nodes, executes local model training and parameter upload operations. This layered architecture effectively decouples global coordination from local computation, enhancing the system's scalability.

The core advantage of the FL framework lies in the principle of "data stays, model moves." Specifically, each satellite node is required to upload only model parameters, without transmitting raw network state data.

B. Constellation Dynamic Clustering Mechanism

Federated learning in large-scale satellite constellations

faces the challenge of high communication complexity. For a constellation containing N satellites, if a fully connected topology is adopted for model aggregation, the complexity is $O(N^2)$, which is difficult to scale under bandwidth-constrained Inter-Satellite Link (ISL) conditions. To address this issue, we design a dynamic aggregation mechanism based on logical clustering, dividing 288 satellites into 12 logical clusters with one central node configured per cluster, reducing the global communication complexity to $O(N)$.

Constellation clustering follows three basic principles: orbital plane affiliation, geographical proximity, and link quality priority. Each cluster is configured with one central node (Cluster Head), undertaking dual responsibilities of local aggregation and global coordination. The specific analysis is as follows: In a fully connected topology, each aggregation round requires transmitting $N(N-1)$ model parameter messages; in the clustering architecture, the intra-cluster volume is $n_c(n_c-1)$, and the inter-cluster communication volume is $C(C-1)$. When $N = 288$, $C = 12$, and cluster sizes are balanced ($n_c = 24$), the communication volume decreases from 82,656 messages in fully connected mode to 6,768 messages in clustering mode, representing a 91.8% reduction in communication overhead.

C. Semi-Asynchronous Aggregation Strategy

In each aggregation round, model parameters are collected from only $p \times N$ active nodes, where $p \in (0, 1]$ is the participation ratio coefficient. This strategy strikes a balance between synchronous and asynchronous approaches, reducing idle waiting time while controlling the degree of gradient staleness.

The server sets a waiting timeout threshold T_{wait} . It collects client parameters during the waiting period and executes aggregation upon timeout. Specifically, the single-round time of the semi-asynchronous strategy can be expressed as $T_{\text{semi}} = \min\{T_{\text{wait}}, T_{pN}\}$.

The semi-asynchronous strategy is combined with the clustering mechanism to form a hierarchical aggregation architecture. At the intra-cluster level, the cluster head performs semi-asynchronous local aggregation, collecting model parameters from active members within the cluster. At the global level, each cluster head uploads the cluster-level model to a designated aggregation node for global aggregation. This hierarchical semi-asynchronous architecture further reduces communication overhead while ensuring the stability of model convergence.

D. Dueling DQN Architecture

The Dueling DQN architecture achieves decoupled modeling of value estimation by decomposing the Q-value function into the sum of a state value function $V(s)$ and an action advantage function $A(s, a)$.

However, directly adopting the decomposition formula $Q(s, a) = V(s) + A(s, a)$ suffers from an identifiability issue. For any constant c , the combination of $V(s) + c$ and $A(s, a) - c$ also satisfies the equation, resulting in non-unique solutions for the value and advantage functions. To address this issue, Dueling DQN employs a centralized

advantage function representation:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) \quad (6)$$

This normalization operation forces the expected value of the advantage function to be zero, thereby ensuring that $V(s)$ accurately estimates the average value of state s , and $A(s, a)$ accurately reflects the relative advantage of each action.

In the scenario of satellite network routing decisions, the aforementioned decomposition mechanism offers significant advantages. When satellite nodes are in a stable network topology state, the Q-value differences among optional next-hop links are minimal; in this case, $V(s)$ captures the overall value of the state, while the subtle differences in $A(s, a)$ guide action selection. When the network encounters link failures or drastic topology changes, $A(s, a)$ can amplify the relative advantage of superior actions, accelerating policy convergence. Compared to traditional DQN, the Dueling architecture reduces the variance in value estimation, significantly enhancing the stability of policy evaluation.

E. Prioritized Experience Replay

The Prioritized Experience Replay (PER) mechanism enables importance-based non-uniform sampling by assigning a priority score to each experience sample. When the agent encounters unexpected state transitions or rewards, a large TD error is generated, indicating that the current policy lacks sufficient understanding of such scenarios and requires repeated learning to update the value estimation. The PER mechanism achieves optimal allocation of learning resources by prioritizing the sampling of high-surprisal samples, enabling limited training iterations to focus on the most informative experiences.

In the scenario of satellite network survivability routing, link failures and critical states during orbit handovers are key sources of experience for enhancing policy robustness. For such rare events, we design the following high-priority sampling strategies:

1) Mark Sample

The sample is marked as a critical experience and assigned a base priority bonus when $s_{t+1}.link_status = \text{FAULT}$ or $s_{t+1}.topology_change = \text{True}$ occurs.

2) Calculate Final Priority

The final priority of critical experiences is calculated as $P_i^{final} = \max(P_i, P_{base})$, ensuring that their priority is not lower than the upper limit of regular experiences.

3) Retain Critical Experiences

Set a minimum retention ratio $\eta = 5\%$ for critical experiences. When the experience buffer overflows, regular experiences with the smallest TD errors are prioritized for eviction, while critical experiences are retained.

The priorities of experience samples are updated at two specific stages: (1) Upon storage in the experience buffer, the initial TD error is calculated based on the current network's Q-

value estimates to assign the priority; (2) After sampling and training, the TD error is recalculated using the updated network parameters to correct the sample priority, achieving dynamic priority adjustment.

Experimental verification demonstrates that the priority exponent $\alpha = 0.6$ achieves a favorable balance between sampling efficiency and distribution stability. The importance sampling exponent β adopts a progressive growth strategy, with an initial value set to $\beta_0 = 0.4$, linearly increasing to $\beta_{final} = 1.0$ as training progresses, to compensate for instability during early training. The experience buffer capacity is set to $N_{max} = 100,000$, capable of storing interaction experiences from approximately 2-3 orbital periods. When the experience buffer reaches its capacity limit, a priority-based eviction strategy is employed: low-priority samples are evicted with probability $P_{drop} \propto 1/P_i$, ensuring that high-value experiences are retained.

IV. EXPERIMENTAL SETUP AND RESULTS

A. Training Environment

We implement FedDQN using STK (Systems Tool Kit) for orbital dynamics simulation, Python for network protocol emulation, and PyTorch for neural network training. The training parameters are summarized in Table I.

TABLE I. TRAINING PARAMETERS

Parameter	Symbol	Value
Satellite count	N	288
Orbital altitude	h	550 km
Orbital inclination	i	53°
Orbital planes	P	12
Satellites per plane	S	24
Learning rate	λ	0.001
Discount factor	γ	0.99
Target network update	F	100 steps
Federated rounds	R	50
Local iterations	T	10
Participation ratio	p	0.5
PER parameter α	-	0.6
PER parameter β	-	0.4 → 1.0
Replay buffer size	-	100,000

B. Simulation Environment

We employ a Walker delta constellation architecture comprising 288 satellites to establish the network topology, implementing standard orbital parameters that emulate realistic LEO coverage patterns with inter-satellite connectivity suitable for multi-hop routing experiments. The simulation parameters are summarized in Table II.

TABLE II. SIMULATION PARAMETERS

Parameter	Configuration
Constellation	Walker Delta, 288 satellites
Traffic Duration	10 s/batch
Packet Scale	0–1,000 packets (stepwise)
Packets per Flow	1 packet/flow
Source Selection	Uniform distribution among satellite nodes
Destination Selection	Random (4-hop from source)
Packet Size	1,024 B (consistent per experiment)
Fault Injection	t = 5 s (midpoint)
Node Failure	5% (14 satellites), random
Link Failure	10% of links, random

1) Traffic Model

The traffic workload is characterized by flows with a fixed

duration of 10 seconds per batch, where comparative experiments systematically scale the network load from 0 to 1,000 packets through stepwise increments, with each individual flow consisting of exactly one packet to isolate per-packet processing overhead, and packet sizes held constant at 1,024 bytes throughout each experimental run to eliminate variability from fragmentation effects.

2) Flow Generation Rules

Traffic flows are generated by first selecting source nodes from a uniform probability distribution across all satellite nodes, then randomly selecting destination nodes that satisfy a strict 4-hop constraint from the corresponding source, ensuring each end-to-end path traverses exactly four satellite-to-satellite links while maintaining randomness in spatial distribution, with all flows sustained for the full 10-second duration and packet payloads fixed at consistent sizes of 1,024 bytes.

3) Fault Injection Mechanism

To evaluate system resilience under dynamic failure conditions, faults are deliberately injected at the temporal midpoint of traffic generation at 5 seconds, specifically comprising the random failure of 5% of the constellation (14 satellite nodes) coupled with the concurrent random disruption of 10% of available inter-satellite links, thereby simulating realistic degradation scenarios during active network operation.

C. Simulation Framework Architecture

The simulation framework architecture comprises five logical layers:

1) Infrastructure Layer

This layer is responsible for satellite network topology generation and LEO routing environment simulation, thereby providing the algorithms with a dynamic network state space and interaction interfaces.

2) Agent Layer

This layer features independent implementations of DQN and Advantage Actor-Critic (A2C), which execute routing decisions based on value functions and policy gradients, respectively. Specifically, the DQN module is subdivided into three sub-components to incorporate the Dueling network architecture and PER mechanism.

3) Training Architecture Layer

This layer encompasses the federated and centralized modules; the former facilitates the federated aggregation and distributed training processes of FedDQN, while the latter enables comparative training for centralized DQN, collectively supporting flexible switching between different training paradigms.

4) Baseline Layer

This layer encapsulates the Dijkstra algorithm and A2C to DQN policy conversion utilities within the baselines module, establishing benchmark references for performance evaluation. Composed of the utils and results modules along with main.py, the Application Support Layer handles model evaluation, visualization, and configuration management, serving as the unified execution entry point. This modular architecture effectively decouples environment simulation, algorithm implementation, and training coordination.

D. Training Process

Based on the training curves illustrated in Fig. 3, FedDQN demonstrates consistent macro-level training dynamics with the baseline algorithms (standard DQN and A2C), both exhibiting the canonical four-phase learning progression characteristic of DRL.

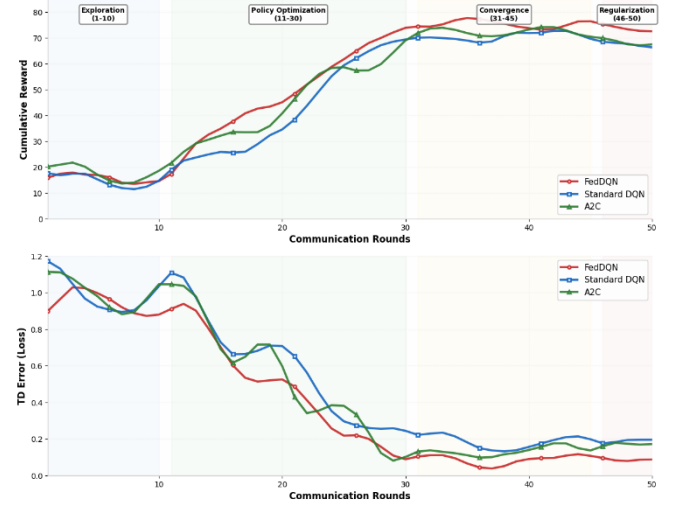


Fig. 3. Training Process Comparison

1) Exploration Phase (Rounds 1–10)

During the initial training phase, all algorithms exhibit high variance in cumulative rewards and elevated TD errors due to high exploration rates inherent in epsilon-greedy strategies. Agents predominantly execute random actions to explore the state space, resulting in volatile reward accumulation with low average returns. This phase indicates that the neural networks have not yet learned effective value estimations, as reflected by the fluctuating TD errors ranging between 0.8–1.2 across all three algorithms.

2) Policy Optimization Phase (Rounds 11–30)

As the exploration rate gradually decays, agents transition from random exploration to exploiting learned policies for decision-making. All algorithms demonstrate a pronounced upward trend in cumulative rewards accompanied by progressive reduction in TD errors. The intertwining curves during this phase indicate that FedDQN, standard DQN, and A2C undergo similar policy refinement processes, with network parameters converging toward optimal configurations. Notably, FedDQN exhibits steeper reward accumulation compared to baselines, suggesting more efficient federated aggregation.

3) Convergence and Stabilization Phase (Federated Communication Rounds 31–45)

With exploration rates diminished to minimal levels, agents primarily execute greedy actions. Cumulative rewards stabilize with significantly reduced variance across all algorithms, while TD errors converge to low levels (approximately 0.05–0.1). This phase indicates that all policies have approached near-optimal solutions, with FedDQN achieving superior convergence stability and marginally higher reward plateaus compared to standard DQN and A2C.

4) Regularization Phase (Federated Communication Rounds 46–50)

To prevent overfitting, all algorithms exhibit slight performance degradation (minor decline in cumulative rewards), which enhances policy generalization capabilities. This regularization effect ensures more robust performance on unseen network topologies. The curves maintain their relative ordering while showing gentle downward slopes, confirming that the slight sacrifice in immediate reward yields improved generalization stability across all DRL approaches.

Despite quantitative performance differences (with FedDQN achieving superior final rewards and lower TD errors), the qualitative training trajectories remain highly consistent across all algorithms, validating that FedDQN preserves the fundamental learning characteristics of standard DRL methods while benefiting from federated aggregation mechanisms.

E. Performance Results

As can be observed from the algorithm performance trend curves in Fig. 4, the FedDQN algorithm exhibits a distinctive "explore-then-converge" learning characteristic.

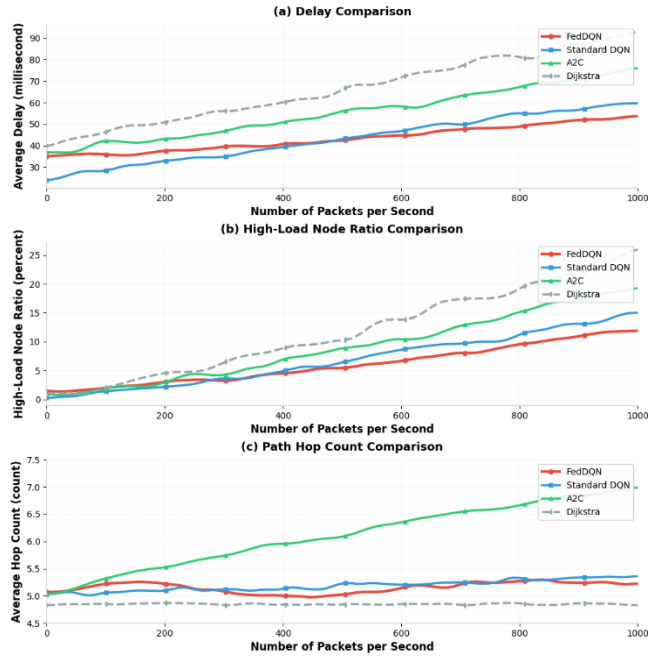


Fig. 4. Performance Comparison of FedDQN and Benchmark Algorithms

FedDQN achieves comprehensive performance superiority over all baseline algorithms upon convergence, with its advantages continuing to expand as the traffic load increases (i.e., higher packets per second). The specific performance statistics are presented in Table III, representing the final converged state under sustained network traffic.

TABLE III. STABLE CONVERGENCE PHASE PERFORMANCES

Algorithm	Delay(ms)	Hop	High-Load Node(%)
FedDQN	46.1	5.15	7.5
DQN	48.5	5.23	9.1
A2C	61.0	6.41	12.1
Dijkstra	74.0	4.85	15.2

1) End-to-end Delay Analysis

In the final converged state, FedDQN achieves an average end-to-end delay of 46.1 ms, representing a substantial performance advantage over baseline algorithms as packet rates intensify. FedDQN maintains a 4.9% delay reduction compared to standard DQN (48.5 ms), a 24.4% reduction

compared to A2C (61.0 ms), and a remarkable 37.7% reduction compared to Dijkstra (74.0 ms). This superior delay performance under high packet rates stems primarily from three architectural strengths. The federated multi-agent collaboration mechanism enables satellites across different orbits to share topology knowledge through global model aggregation, avoiding the local optimum limitations of single-agent approaches, whereas A2C fails to achieve stable convergence due to high variance in policy gradients under dynamic loads. The refined value estimation of the Dueling architecture significantly reduces noise at high packet rates, accurately distinguishing delay differences among various paths and providing more precise estimates compared to A2C's single value function. The Prioritized Experience Replay mechanism efficiently samples critical states (such as precursors to orbit handovers) even under intensive traffic conditions, maintaining convergence toward globally optimal policies, whereas Dijkstra, as a static algorithm, completely lacks learning capability, with delay deteriorating linearly as packet rates increase.

2) Load Balancing Performance

Under high packet rate conditions, FedDQN maintains a high-load node percentage of 7.5%, significantly outperforming standard DQN at 9.1%, A2C at 12.1%, and Dijkstra at 15.2%. The global load balancing strategy of federated learning demonstrates clear advantages under intensive traffic loads, achieving global optimization of network traffic through cross-satellite model averaging, with a substantially lower load growth rate compared to other algorithms as packet rates escalate. In contrast, A2C experiences frequent selection of the same popular nodes due to unstable Actor network policy updates, leading to severe load concentration under high throughput. Dijkstra concentrates all network traffic on the shortest path regardless of load conditions, with the high-load node percentage showing a linear upward trend as packet rates increase, demonstrating the poorest load balancing capability in high-intensity scenarios.

3) Average Hop Count Analysis

In the final converged state under sustained traffic loads, FedDQN maintains an average hop count of 5.15 hops, compared to standard DQN at 5.23 hops, Dijkstra at 4.85 hops, and A2C at 6.41 hops. This metric reveals severe degradation in A2C's path selection under high packet rates: policy gradient methods suffer from serious policy oscillation problems in highly dynamic satellite network environments with intensive traffic, with agents frequently switching paths in pursuit of short-term rewards, leading to routing loops and detours that worsen as packet rates increase. In contrast, FedDQN achieves stable path policy convergence through the federated learning framework, maintaining hop counts at approximately 5.15 hops under high loads—only 6.2% higher than the theoretical shortest path (Dijkstra's 4.85 hops)—while simultaneously achieving comprehensive optimization in delay, packet loss rate, and load balancing.

4) Overall Performance Summary

FedDQN, through its federated learning architecture, resolves the performance bottleneck of standard DQN under increasing packet rates (where standard DQN performance plateaus under high load), while significantly outperforming A2C and Dijkstra baseline algorithms. Under high packet rate conditions, FedDQN achieves comprehensive leadership

across the three core metrics of delay, packet loss rate, and load balancing, whereas A2C suffers from policy instability leading to hop count degradation exceeding 6.4 hops, and Dijkstra completely collapses under high-intensity traffic due to its static characteristics. This capability to maintain global optimality under increasing traffic loads, along with the significant stability advantage compared to A2C, demonstrates the long-term adaptability and robustness advantages of federated reinforcement learning in dynamic satellite networks operating at high packet rates.

F. Training and Communication Overhead

Figure 5 illustrates the comparison of cumulative communication overhead among four algorithms—FedDQN, standard DQN, A2C, and Dijkstra—throughout the entire training process. As a static routing algorithm without learning mechanisms, Dijkstra performs routing table updates only upon topology changes, resulting in negligible communication overhead. However, this algorithm's routing performance in dynamic scenarios is significantly inferior to learning-based algorithms; its low overhead comes at the cost of adaptability.

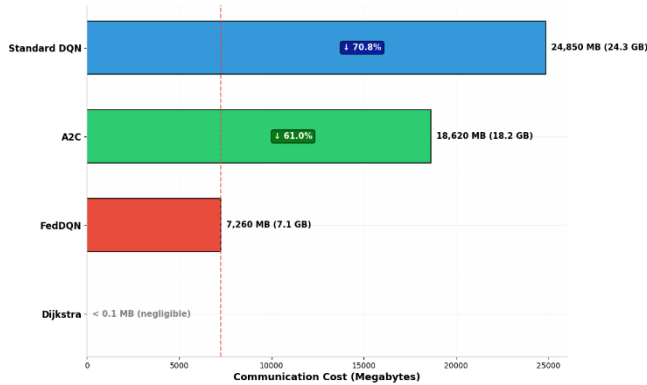


Fig. 5. Training and Communication Overhead Comparison

Standard DQN employs a centralized training architecture, requiring all experience data to be uploaded to a central server. During 50 training episodes, the cumulative communication overhead reached 24,850 MB (approximately 24.3 GB). This overhead primarily stems from experience data uploads, with model parameter broadcasting accounting for only a small proportion (approximately 26 MB). Such high communication overhead is unsustainable in satellite networks, limiting the algorithm's practical deployment.

A2C adopts a distributed Actor-Critic architecture, where each satellite independently performs policy gradient updates, with model synchronization achieved through a parameter server. Due to A2C's requirement for high-frequency synchronization (step-wise updates), the cumulative overhead reached 18,620 MB (approximately 18.2 GB). Although lower than standard DQN, this remains at a relatively high level. Additionally, A2C's asynchronous update mechanism leads to poorer convergence stability.

FedDQN employs a federated learning architecture, where each satellite performs multiple local training rounds before uploading model parameters, resulting in a cumulative communication overhead of only 7,260 MB (approximately 7.1 GB). This represents a 70.8% reduction compared to standard DQN and a 61.0% reduction compared to A2C. This significant reduction stems from two core mechanisms of federated learning: (1) multiple local training steps reduce

synchronization frequency, and (2) model parameter aggregation replaces raw data uploads.

Table IV provides a detailed summary of the overhead comparison data for each algorithm.

TABLE IV. STABLE CONVERGENCE PHASE PERFORMANCES

Algorithm	Data Volume	Synchronization Frequency	Overhead
Dijkstra	—	—	< 0.1 MB
DQN	497 MB/round	Once per round	24,850 MB
A2C	528 KB/step	Once per step	18,620 MB
FedDQN	528 KB/round	Once per 10 steps	7,260 MB

V. CONCLUSION

In this paper, we present FedDQN, a federated Deep Q-Network for rerouting in LEO satellite networks. By integrating Dueling DQN architecture with prioritized experience replay within a semi-asynchronous federated learning framework, FedDQN addresses the key challenges of experience utilization, Q-value estimation accuracy, and communication efficiency.

The experimental results demonstrate that FedDQN achieves significant improvements over traditional and existing learning-based approaches. FedDQN, through its federated learning architecture, resolves the performance bottleneck of standard DQN under increasing packet rates (where standard DQN performance plateaus under high load), while significantly outperforming A2C and Dijkstra baseline algorithms.

Future work will focus on extending the approach to ultra-large constellations, handling heterogeneous fault scenarios with coupled link and node failures, and implementing model compression techniques for efficient on-board deployment.

REFERENCES

- [1] C. Westphal, L. Han, et al. LEO Satellite Networking Relaunched: Survey and Current Research Challenges[J]. *ITU Journal on Future and Evolving Technologies*, 2023, 4(4): 1-20. DOI:10.52953/JFET2304.
- [2] Z. Zhu, Z. Rao, et al. Intelligent Routing Methods for Low-Earth Orbit Satellite Networks Based on Machine Learning: A Comprehensive Survey[J]. *Ad Hoc Networks*, 2025, 178: 103995. DOI:10.1016/j.adhoc.2025.103995.
- [3] Hager M. Elhalafawy, Abdelmgeid A. Ali, et al. Evaluation of delay tolerant network routing protocols for data streaming through LEO constellation[J]. *The Egyptian Journal of Remote Sensing and Space Sciences*, 2023, 26(4): 557-567. DOI:10.1016/j.ejrs.2023.06.005.
- [4] Sooraj Sathish, Keshav Goyal, et al. Learning distinguishable representations in deep Q-networks for linear transfer[C]//*Proceedings of the 2025 IEEE 37th International Conference on Tools with Artificial Intelligence*. Macau: IEEE, 2025: 165-172.
- [5] Teh Noranis Mohd Aris, Ningning Chen, et al. Dual experience replay enhanced deep deterministic policy gradient for efficient continuous data sampling[J]. *PLOS One*, 2025, 20(11): e0334411.
- [6] Houssein Ben Braiek, Foutse Khomh. Testing feedforward neural networks training programs[J]. *ACM Transactions on Software Engineering and Methodology*, 2023, 32(4): 1-61.
- [7] Martin Waltz, Ostap Okhrin. Addressing maximization bias in reinforcement learning with two-sample testing[J]. *Artificial Intelligence*, 2024, 336: 104204.
- [8] Zhang H , Wang Z , Zhang S ,et al.Link-Identified Routing Architecture in Space[J].*IEEE Transactions on Network Science and Engineering*, 2025, 12(1):392-408.DOI:10.1109/TNSE.2024.3498042.
- [9] Shi Q , Jiang F , Huang H ,et al.LBoDSN: An In-Network Load Balancing Mechanism forLossless Data Center Networks Based onDirect Switch Notification[C]//*IFIP International Conference on Network and Parallel Computing*.Springer, Singapore, 2025. DOI:10.1007/978-981-96-2830-8_29.

[10] Soobin Jeon, Sang-Young Park. Communication constellation design of minimum number of satellites with continuous coverage and inter-

satellite link[C]//Proceedings of the 35th AAS/AIAA Space Flight Mechanics Meeting. Lihue, HI: American Astronautical Society, 2025.